LORDS SCHOOL OF ENGINEERING & TECHNOLOGY,

ALWAR

A MINOR PROJECT SYNOPSIS

ON

ONLINE TAXI BOOKING & FLEET DISPATCH SYSTEM FOR A TOURIST TAXI OPERATOR

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY IN

COMPUTER SCIENCE AND ENGINEERING

SUBMITTED TO:
Dr. Pratap Singh Patwal
Head of Department of CSE

SUBMITTED BY:
Student Name: Krish
Roll No: 4133

Academic Session 2026-2027

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

TABLE OF CONTENTS

1. ABSTRACT **3**

2. INTRODUCTION & OBJECTIVES **4**

2.1 Introduction 4

2.2 Objectives of the Project 4

3. BLOCK DIAGRAM OF THE SYSTEM **5**

4. METHODOLOGY & SYSTEM DESIGN **6**

4.1 Methodological Flowchart 6

4.2 Booking & Fare Estimation Module 6

4.3 Persistence & Notification Module 7

4.4 Dispatch Dashboard & Reporting 7

4.5 Data Design & Schema 8

5. TOOLS & TECHNOLOGIES USED **9**

6. ADVANTAGES AND LIMITATIONS **9**

6.1 Advantages 9

6.2 Limitations & Constraints 10

7. APPLICATIONS OF THE PROJECT **10**

8. REFERENCES & BIBLIOGRAPHY **10**

1. ABSTRACT

Small and medium taxi operators in tourist regions still run their business on the telephone. A booking is noted in a diary or a chat window, the fare is calculated verbally from a per-kilometre rate, and the customer receives no written confirmation. There is no way to check whether a driver has actually been assigned, and no record to fall back on if the trip is disputed. The owner, in turn, has no single register of the day's trips and no quick way to prepare a billing sheet.

This project presents **Gaurav Tourist & Taxi Services**, a lightweight online taxi booking and fleet dispatch system built for such an operator. The full application is a single responsive web page written in HTML5, CSS3 and plain JavaScript, and it needs no application server, no build tool and no paid hosting. The public site offers three service types — local city rides, outstation one-way or round trips, and multi-day tour packages — and the booking form changes its fields according to the service selected.

Pickup and drop-off points are captured through Google Places Autocomplete, and the Google Distance Matrix service returns the actual driving distance and travel time between them. The distance is filled into the form automatically and the fare is calculated live from the selected vehicle's rate card, so the customer sees a transparent estimate before confirming. Every confirmed booking is given a unique code in the form GT-XXXX, written to a Google Sheet and pushed to the owner's WhatsApp as a formatted message, with an SMS link as a fallback.

A password-protected owner dashboard lists all bookings, allows a driver to be assigned, moves a trip through the pending, assigned, completed and cancelled states, and exports the whole register to an Excel workbook using SheetJS. The customer can check the status of a booking from the same site at any time by entering the booking code. The result is a zero-cost, mobile-first booking channel in which the customer relationship stays with the operator instead of an aggregator.

2. INTRODUCTION & OBJECTIVES

2.1 Introduction

Alwar and the surrounding region receive a steady flow of tourists, pilgrims and business travellers, and a large share of local transport is provided by owner-driven taxis operating two to eight vehicles. Aggregator applications bring customers to these operators, but they charge a commission of roughly eighteen to twenty-five per cent on every trip, control the customer's contact details and rank the operator against its own competitors. Operators therefore prefer direct bookings, yet their direct channel is little more than a phone number painted on the rear glass of the car.

Direct booking today has three weak points. The fare is quoted from memory and argued about at the end of the trip; the booking exists only in a call log, so two customers can be promised the same vehicle; and the traveller has no way to know whether a driver has been assigned. A written, code-stamped confirmation with a live status field removes all three problems, and a single shared register removes the double-booking risk.

The usual answer is a server application with a relational database, but that brings a monthly bill, database administration and a deployment pipeline that a small operator cannot maintain. Since the business already runs on a Google account, Google Sheets can act as the data store and Google Apps Script as the web endpoint. The front end then stays a static file that can be hosted free of cost, and the owner edits nothing more complicated than a spreadsheet.

2.2 Objectives of the Project

- **Adaptive booking interface:** Provide one form that serves local rides, outstation trips and tour packages, revealing only the fields relevant to the selected service.
- **Transparent fare estimation:** Replace verbal quoting with an automatic driving distance from Google Maps and a fare computed from a fixed per-kilometre rate card for each vehicle.
- **Written confirmation and tracking:** Issue a unique booking code (GT-XXXX) on submission and let the customer retrieve the live status of the trip from the public site.
- **Instant owner notification:** Deliver the complete booking to the operator's phone through a WhatsApp deep link, with a platform-aware SMS link when WhatsApp is unavailable.
- **Dispatch and status control:** Give the owner a password-protected dashboard to assign a driver and move each booking through pending, assigned, completed and cancelled states.
- **Zero-cost deployment and reporting:** Persist bookings in a Google Sheet without any server, and export the full register to Excel for billing and accounting.

3. BLOCK DIAGRAM OF THE SYSTEM

The system follows a four-layer model: the customer's browser, a static web application, a serverless Apps Script endpoint that fronts the spreadsheet, and the Google Sheets store itself. Google Maps and the owner dashboard attach to the application layer as supporting services.

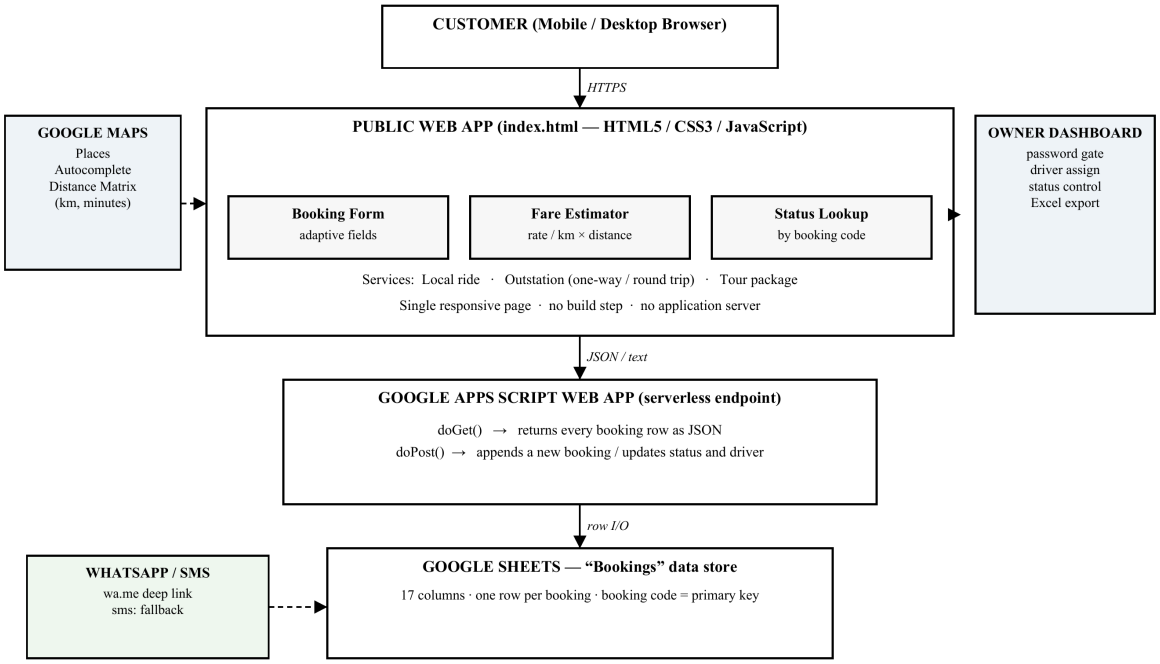


Figure 1: Block diagram of the proposed taxi booking and dispatch system

4. METHODOLOGY & SYSTEM DESIGN

4.1 Methodological Flowchart

The working of the system from the customer's first click to the owner's dispatch action is shown below. Steps 1 to 5 run in the browser, step 6 crosses into the Apps Script endpoint, and steps 7 and 8 complete the loop through notification and dispatch.



Figure 2: Methodological flowchart of the proposed system

4.2 Booking & Fare Estimation Module

This module owns everything that happens before a booking is submitted. It adapts the form to the chosen service, resolves place names into coordinates and converts distance into a price.

- **Service selection:** The functions setService() and the service tabs switch between *local*, *outstation* and *tour*. Drop-off, trip type, tour duration, the distance field and the estimate panel are shown or hidden accordingly, and the drop-off field is made optional only for tour packages.
- **Address capture:** Google Places Autocomplete is attached to the pickup and drop-off inputs and returns the place geometry. Editing the text after choosing a suggestion invalidates the stored place, so a stale coordinate can never be priced.
- **Distance calculation:** When both places are known, DistanceMatrixService is called in DRIVING mode with the metric unit system. The returned distance in metres is converted to kilometres, rounded to one decimal place and written into a read-only field along with the travel time. If the API fails, the field becomes editable so the operator can enter the distance manually.
- **Rate card and live estimate:** Each vehicle option carries a data-rate attribute — Swift Dzire and Etios at ₹20 per km, Ertiga at ₹25 and Innova Crysta at ₹30. The estimate is

computed as *round(rate × distance)* and is recalculated whenever the vehicle or the distance changes. Outstation and tour bookings instead show a note that the fare will be quoted after contact.

Core of the fare estimator (index.html):

```
function updateEstimate() {
  const rate = parseFloat(vehicle.selectedOptions[0].dataset.rate);
  const km   = parseFloat(distance.value);
  if (!km || km <= 0) { estimateBox.classList.add("hidden"); return; }
  const total = Math.round(rate * km);
  estimateAmount.textContent =
      "₹" + total.toLocaleString("en-IN");
  estimateDetail.textContent =
      vehicle.value + " · " + km + " km × ₹" + rate + "/km";
  estimateBox.classList.remove("hidden");
}
```

4.3 Persistence & Notification Module

Once confirmed, a booking must survive a page refresh, reach the owner and stay queryable by the customer. A Google Sheet behind an Apps Script web app provides all three without a server.

- **Booking record:** On submit the form is serialised into a JSON object holding the generated code, service type, name, phone, pickup, drop-off, trip type, duration, vehicle, distance, estimated fare, date, time, notes, status (set to *pending*), an empty driver field and an ISO-8601 creation timestamp.
- **Serverless endpoint:** The record is posted as *text/plain* to a Google Apps Script web-app URL. The doPost() handler parses the payload and appends one row to the Bookings sheet, while doGet() returns every row as a JSON array that the site reads back for lookups and for the dashboard.
- **Confirmation and notification:** The stub card on the page displays the booking code and its status. A message containing the route, vehicle, fare, date and notes is URL-encoded into a wa.me link that opens the operator's chat with the text pre-filled; a second link builds an sms: URI using the separator expected by the customer's platform, so the booking still reaches the owner on a device without WhatsApp.
- **Status lookup:** The public lookup box accepts a booking code, matches it case-insensitively against the sheet, and renders a ticket showing the route, schedule, vehicle, fare, assigned driver and current status.

4.4 Dispatch Dashboard & Reporting

- **Access control:** The dashboard stays behind a password prompt; the entered value is compared with the dispatch constant before any record is fetched, so ordinary visitors never load the booking list.
- **Dispatch board:** Every booking is rendered as a card showing its code, customer name and route, together with a driver-name input and a status selector offering pending, assigned, completed and cancelled.

- **Status update:** The save action posts a small JSON payload — *{action: "update", id, status, driver}* — to the same Apps Script endpoint. The handler locates the row by booking code and rewrites only the status and driver cells, so the customer's next lookup reflects the change immediately.
- **Excel export:** After the same password check, SheetJS converts the booking array into a seventeen-column worksheet and writes it as *gaurav-taxi-bookings-YYYY-MM-DD.xlsx*. The owner thus receives a dated, accounting-ready file without copying anything by hand.

4.5 Data Design & Schema

Each booking is stored as one row of the Bookings sheet. The columns below form the complete record; the booking code is generated by the browser and is unique for every trip.

Field	Type	Description
Booking Code	Text (GT-XXXX)	Unique code generated on submit; primary key of the sheet.
Service	Enum	local, outstation or tour.
Customer Name	Text	Name entered in the booking form.
Phone	Text	Contact number used for confirmation and callback.
Pickup	Text	Pickup place, resolved through Autocomplete.
Drop-off	Text	Destination place or city.
Trip Type	Enum	one-way or round-trip (outstation only).
Duration	Text	e.g. “3 days / 2 nights” (tour only).
Vehicle	Enum	Swift Dzire, Ertiga, Innova Crysta or Etios.
Distance (km)	Decimal	Driving distance from the Distance Matrix API.
Estimated Fare	Integer	Rate per km × distance, rounded (local rides).
Date / Time	Date, Time	Requested pickup schedule.
Notes	Text	Passengers, luggage and vehicle preference.
Status	Enum	pending, assigned, completed or cancelled.
Driver	Text	Driver assigned by the owner from the dashboard.
Created At	Timestamp	ISO-8601 stamp written when the booking is saved.

The data store is deliberately flat: one sheet, one row per booking, and the generated booking code as the primary key. Because the sheet is maintained by Apps Script and read by the same code that writes it, no joins or foreign keys are required; the structure is equivalent to a single-table schema with enumerated status and service columns.

5. TOOLS & TECHNOLOGIES USED

The stack was chosen so that every layer is free, browser-based and maintainable by a single operator without a server administrator.

Category	Technology / Library	Primary Function in the System
Front-end Markup	HTML5	Semantic single-page structure: hero, service cards, booking form and owner panel.
Styling	CSS3 — custom properties, Flexbox, Grid, media queries	Responsive “ticket” interface with a fixed colour and type system.
Programming Language	JavaScript (ES6+)	Form state, fare engine, asynchronous fetch calls, DOM rendering and validation.
Geo Services	Google Maps JavaScript API — Places Autocomplete, Distance Matrix	Address suggestions and automatic driving distance and travel time.
Serverless Backend	Google Apps Script Web App (doGet / doPost)	REST-style endpoint that reads and writes booking records.
Data Store	Google Sheets	Cloud spreadsheet holding one row per booking, keyed by booking code.
Notification	WhatsApp Click-to-Chat (wa.me), sms: URI	Instant booking alert to the operator with an SMS fallback.
Reporting	SheetJS (xlsx) 0.18	Builds the downloadable Excel register of all bookings.
Hosting	Static hosting / GitHub	Cost-free deployment; no server, database or build pipeline.

6. ADVANTAGES AND LIMITATIONS

6.1 Advantages

- **Zero infrastructure cost:** the only running components are a static page, a Google Sheet and an Apps Script trigger, all of which fit inside a free Google account.
- **Transparent pricing:** the fare is derived from a published rate card and the actual driving distance, which removes the argument that usually begins at the end of a trip.
- **One register, no double entry:** the customer's submission, the owner's dispatch board and the Excel export all read the same sheet, so no booking has to be copied by hand.
- **Low-bandwidth and mobile-first:** the entire application is a single small file that loads on an entry-level phone over a weak connection, which matters for customers booking from a highway or a railway platform.
- **Immediate notification:** the WhatsApp message reaches the operator within seconds of submission, without any paid messaging gateway or server-side worker.

6.2 Limitations & Constraints

- **Credentials in client code:** the Maps key, dispatch password and Apps Script URL are visible in the page source. This is acceptable for a single-operator tool, but a public deployment must move the password and write operations behind server-side authentication.
- **Spreadsheet as a database:** Sheets provides no transactions or row-level locking, and Apps Script enforces daily execution quotas. The design therefore suits a small fleet rather than hundreds of concurrent bookings.
- **Notification depends on the device:** wa.me and sms: links are handed to the customer's phone; there is no delivery guarantee and no server-side retry if the message is never sent.
- **Partly manual pricing:** outstation and tour fares are still quoted by the owner after contact, so only local rides are priced end to end by the application.

7. APPLICATIONS OF THE PROJECT

- **Small taxi and tour operators:** the primary use case — an owner with a handful of vehicles who needs a booking register, a fare calculator and a dispatch board on a single phone screen.
- **Hotel, homestay and guest-house transfers:** the same page can be re-branded as the in-house airport and sightseeing booking desk of a small property, with the vehicle list edited in one place.
- **Driver cooperatives and shared fleets:** a group of drivers can share one sheet as a common dispatch board, so a trip refused by one driver is visible to the next.
- **Teaching serverless web development:** the project is a compact, complete example of a static front end, an API endpoint and a cloud data store working together without a traditional backend.

8. REFERENCES & BIBLIOGRAPHY

1. HTML Living Standard, WHATWG, 2026. https://html.spec.whatwg.org/
2. MDN Web Docs — JavaScript Guide, Fetch API and Client-side APIs, Mozilla, 2026. https://developer.mozilla.org/en-US/docs/Web/JavaScript
3. Google Maps JavaScript API — Places Autocomplete and Distance Matrix Service, Google, 2026. https://developers.google.com/maps/documentation/javascript
4. Google Apps Script — Web Apps and Content Service, Google, 2026. https://developers.google.com/apps-script/guides/web
5. SheetJS Community Edition — Spreadsheet Data Toolkit, 2026. https://docs.sheetjs.com/
6. WhatsApp Business Help Centre — Click to Chat, Meta Platforms, 2026. https://faq.whatsapp.com/
7. D. Flanagan, *JavaScript: The Definitive Guide*, 7th ed., O'Reilly Media, 2020.
8. Department of CSE, *Project Guidelines and Formatting Manual*, Lords School of Engineering & Technology, Alwar, 2026.